Three distinct computational goals

Matrix computation must account for the following goals.

1. Speed depends on arithmetic, memory, communication, and parallelism.
2. Accuracy measures distance from the exact solution.
3. Backward stability asks whether rounding effects can be represented by small input perturbations. This differs from the problem's sensitivity to those perturbations.

An exact identity need not give a stable algorithm. Even $\det A = 1$ does not ensure insensitivity to perturbation. A mathematically nonsingular matrix can be poorly conditioned.

Specify the error metric.

$$\text{absolute error } \|x - \widehat{x}\|, \qquad \text{relative error } \frac{\|x - \widehat{x}\|}{\|x\|},$$

$$\text{residual } \|b - A\widehat{x}\|.$$

A claim of small error requires a norm, scale, and tolerance.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Floating point and backward error

The basic floating-point model is

$$\mathrm{fl}(x \circ y) = (x \circ y)(1+\delta), \qquad |\delta| \le u,$$

Here u is unit roundoff. The model assumes represented inputs, ordinary rounding, and no overflow or underflow effects requiring a separate model. Cancellation need not violate the single-operation relative model, but can expose errors already present in the operands. Growth and conditioning also affect accumulated error.

Subtracting nearly equal values leaves few significant digits relative to the original operand scale.

$$1.0000001 - 1.0000000 = 1 \cdot 10^{-7}.$$

Backward error asks whether the computed $\widehat{x}$ exactly solves a nearby problem.

$$(A + \Delta A)\widehat{x} = b + \Delta b.$$

A backward-stable algorithm produces small relative input perturbations, but an ill-conditioned problem can still have a large forward error.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Elimination multipliers

For pivot d and an entry below it, the multiplier is

$$\ell_{ik} = \frac{a_{ik}}{d}.$$

A small pivot can create a large multiplier and amplify rounding effects. Partial pivoting selects a largest-magnitude available entry in the current column.

$$|d| = \max_{i \geq k} |a_{ik}|.$$

This bounds every elimination multiplier by 1 in magnitude. It controls one source of growth but does not guarantee a small growth factor for every matrix. Other pivoting strategies can be useful for particular problems.

The factorization is

$$PA = LU.$$

Apply row permutations consistently to stored multipliers. Swapping rows of the active upper factor while leaving earlier lower-factor multipliers unpermuted gives the wrong factorization.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Factorization and solution cost

For dense square matrices, the leading costs are

- pivoted LU:$\frac{2}{3}n^3 + O(n^2)$ flops;
- forward solve $Ly = Pb$:$O(n^2)$;
- back solve $Ux = y$:$O(n^2)$.

Reuse a factorization for multiple right-hand sides instead of repeating cubic work.

A solve can follow this sequence.

1. Factor with suitable pivoting.
2. Estimate matrix norms and conditioning.
3. solve;
4. Compute the true residual in the original equation.
5. Apply iterative refinement if needed.
6. Record an error estimate and its scale assumptions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Factorization and solution cost (continued)

A nonzero pivot or determinant is insufficient as an accuracy check. Determinants can overflow or underflow, and their magnitudes are not scale-invariant condition measures.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sparse matrices and fill-in

Large engineering matrices are often sparse. Storing n^2 zero entries wastes memory and multiplication work.

Whether elimination preserves zeros depends on ordering. Eliminating a star's center first connects all leaves into a clique; eliminating leaves first reduces fill-in.

Read the sparsity graph as follows.

- Variables are vertices.
- Nonzero off-diagonal entries are edges.
- Eliminating a vertex creates fill edges between its remaining neighbors.

Minimum-degree ordering, nested dissection, and reverse Cuthill–McKee can reduce fill or bandwidth. Parallel systems also depend on communication and load balance.

Fix the small numerical example

$$A = \begin{bmatrix} 0.001 & 1 \\ 1 & 1 \end{bmatrix}, \qquad b = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Sparse matrices and fill-in (continued)

Without a swap, the first multiplier is $1/0.001 = 1000$, giving

$$U = \begin{bmatrix} 0.001 & 1 \\ 0 & -999 \end{bmatrix}.$$

After swapping, the multiplier is only 0.001, giving

$$U = \begin{bmatrix} 1 & 1 \\ 0 & 0.999 \end{bmatrix}.$$

Both exact-arithmetic paths give the same solution

$$x = \left(\frac{1000}{999}, \frac{998}{999} \right)^T, \qquad Ax - b = 0.$$

The star graph separately illustrates ordering. Center-first elimination creates three fill edges between the leaves; leaves-first elimination creates none. Numerical growth and sparse fill are distinct costs.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Sparse matrices and fill-in (continued)

With lower and upper bandwidths p, q, band storage uses $O(n(p+q))$ memory, no-pivot band elimination uses $O(npq)$ work, and triangular solves use $O(n(p+q))$. Tridiagonal matrices have $p = q = 1$, so factorization and solution cost $O(n)$. These bounds assume pivoting does not destroy the bandwidth; otherwise account for permutations and the resulting bandwidth.

Pivoting: multiplier size and the same solution

No row exchange

A=[[0.001,1],[1,1]]

ℓ_{21}=1000

U=[[0.001,1],[0,-999]]

x=(1000/999,998/999) · Ax-b=0

Partial pivoting

Swap the two rows first

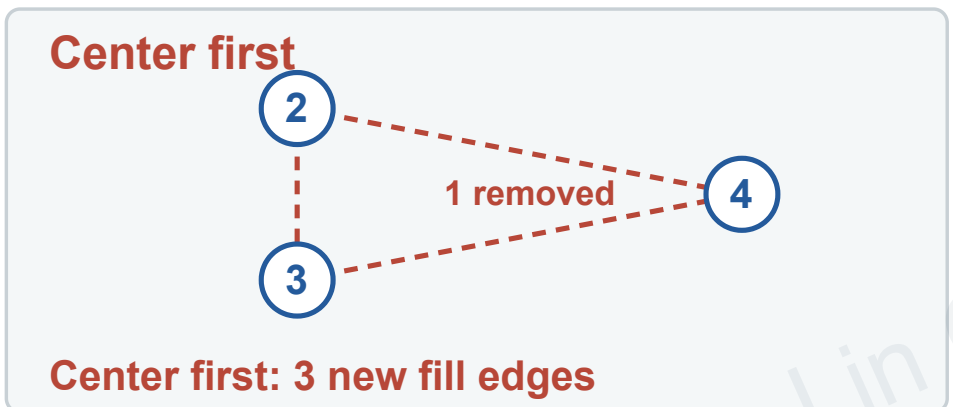
ℓ_{21}=0.001

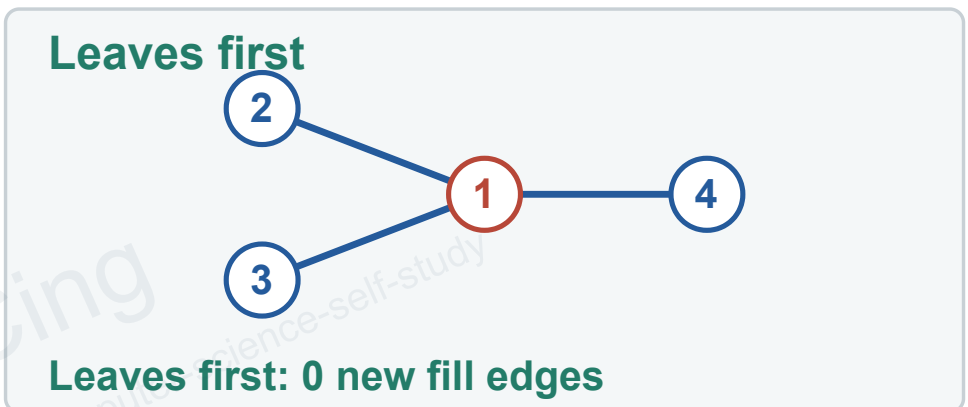
U=[[1,1],[0,0.999]]

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Sparse matrices and fill-in (continued)

Elimination ordering: fill edges in a sparse star





Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Positive-definite elimination and Cholesky

For symmetric positive-definite $A = A^T$, symmetric elimination gives

$$A = LDL^T,$$

with unit lower-triangular L and positive diagonal D. Set

$$R = LD^{1/2}, \qquad A = RR^T$$

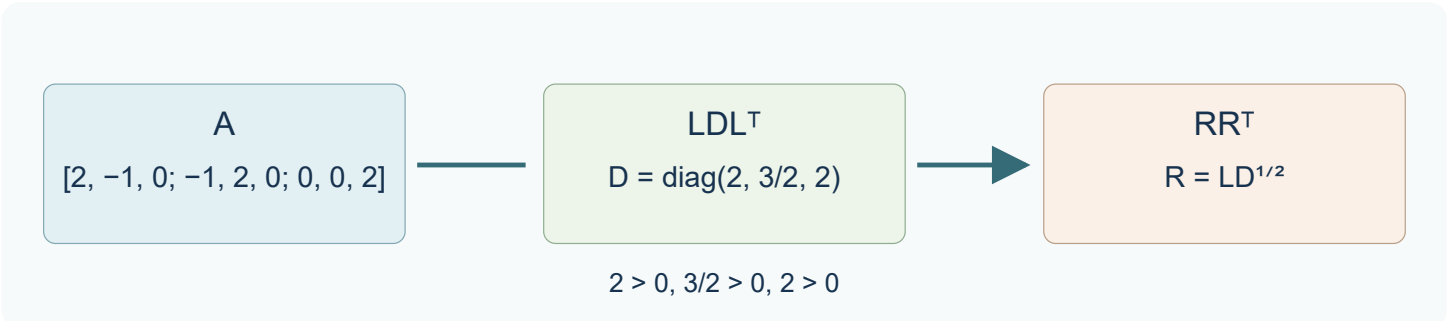
to obtain Cholesky. Storing one triangle requires about $n(n+1)/2$ values, and the leading cost is $n^3/3$, half that of general LU. Existence also follows inductively. The first pivot $a_{11} > 0$, and elimination leaves

$$S = A_{22} - \frac{1}{a_{11}} a_{21} a_{21}^T$$

as a positive-definite Schur complement. For any nonzero y, substitute $[-(a_{21}^T y)/a_{11};\ y]$ into $x^T A x > 0$, obtaining $y^T S y > 0$. Induction gives positive pivots at every step.

For a real symmetric matrix, positive pivots in elimination without row exchanges test positive definiteness. For full-column-rank A, forming $A^T A$ for least squares squares the condition number, $\kappa_2(A^T A) = \kappa_2(A)^2$, and may overflow. Prefer stable QR directly on A.

From elimination to Cholesky: a worked example



Take $A = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & 0 \\ 0 & 0 & 2 \end{bmatrix}$. Eliminating the first column gives the Schur complement $\operatorname{diag}(3/2, 2)$, hence

$$L = \begin{bmatrix} 1 & 0 & 0 \\ -1/2 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad D = \operatorname{diag}(2, 3/2, 2), \quad A = LDL^T.$$

Every entry of D is positive. With $R = LD^{1/2}$, we obtain $A = RR^T$.

$$R = \begin{bmatrix} \sqrt{2} & 0 & 0 \\ -1/\sqrt{2} & \sqrt{3/2} & 0 \\ 0 & 0 & \sqrt{2} \end{bmatrix}, \quad (RR^T)_{22} = 1/2 + 3/2 = 2.$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Stable QR and Householder reflections

A Householder reflector is

$$H = I - 2\frac{vv^T}{v^T v}.$$

For nonzero v, it is symmetric orthogonal, so $H^T H = I$. For nonzero x, choose $\alpha = -\operatorname{sign}(x_1)\|x\|$, with sign 1 at $x_1 = 0$, and $v = x - \alpha e_1$. Since $v^T x = \|x\|^2 - \alpha x_1$, $v^T v = 2(\|x\|^2 - \alpha x_1)$, we obtain $Hx = \alpha e_1$. This sign avoids cancellation in v_1. Successive reflectors give

$$R = H_k \cdots H_1 A, \qquad Q = H_1 \cdots H_k,$$

and hence $A = QR$.

Householder QR generally preserves orthogonality better than classical Gram–Schmidt and is standard for dense least squares.

For full-column-rank $A \in \mathbb{R}^{m\times n}$, $m \geq n$, use the full orthogonal $Q \in \mathbb{R}^{m\times m}$.

$$Q^T A = \begin{bmatrix} R \\ 0 \end{bmatrix}, \qquad Q^T b = \begin{bmatrix} c_1 \\ c_2 \end{bmatrix},$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Stable QR and Householder reflections (continued)

Here R is invertible upper triangular of size $n \times n$. Solve the first n equations.

$$Rx = c_1.$$

For reduced QR, $A = Q_1 R$ with $Q_1 \in \mathbb{R}^{m \times n}$, the formula is $Rx = Q_1^T b$. Full $Q^T b$ has m components and cannot be the right-hand side of n equations without taking its first block.

The minimum residual squared is $\|c_2\|_2^2$. Do not form $A^T A$ or an explicit R^{-1}. Dense Householder QR costs about $2mn^2 - \frac{2}{3}n^3$ flops. Forming Q adds storage and work; a solve can apply the stored reflectors directly.

A Givens rotation equals the identity except in a 2×2 block on coordinates i, j, and can annihilate one selected entry.

$$G_{ij} = \begin{bmatrix} c & s \\ -s & c \end{bmatrix}, \qquad c^2 + s^2 = 1.$$

It preserves orthogonality and permits local parallel operations, useful for sparse or banded structures. Dense methods usually favor Householder reflectors, which clear a column segment at once.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Power iteration and QR eigenvalue iteration

Dense nonsymmetric eigenvalue solvers usually use QR iteration rather than roots of a high-degree characteristic polynomial. First consider power iteration when

$$A = X\,\mathrm{diag}(\lambda_1, \ldots, \lambda_n)X^{-1}, \qquad |\lambda_1| > |\lambda_2| \geq \cdots,$$

and the initial vector has a nonzero coefficient in the leading eigendirection. Then $A^k v_0$ approaches that direction at a rate controlled by $|\lambda_2/\lambda_1|^k$. Normalize each step to prevent overflow or underflow. Tied largest moduli, as in a rotation, need not yield a unique limiting direction.

Inverse iteration solves

$$(A - \mu I)w_{k+1} = w_k$$

and normalizes, amplifying the eigendirection nearest the shift μ. The Rayleigh quotient

$$\mu_k = \frac{w_k^T A w_k}{w_k^T w_k}$$

often supplies the next shift. A nearly singular $A - \mu I$ requires checking solver status; breakdown is not proof of exact convergence.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Power iteration and QR eigenvalue iteration (continued)

This direction-convergence claim assumes diagonalizability, a shift that is not an eigenvalue, a unique eigenvalue nearest μ, and a nonzero initial coefficient in that eigendirection. The transformed eigenvalues $(\lambda_i - \mu)^{-1}$ then satisfy the power method's unique-largest-modulus condition. Equal distances do not guarantee a unique direction. Use real Rayleigh quotients for real symmetric problems and $w^H Aw/(w^H w)$ for complex ones.

The unshifted QR step is

$$A_k = Q_k R_k, \qquad A_{k+1} = R_k Q_k.$$

Since

$$A_{k+1} = Q_k^T A_k Q_k,$$

so every step is a similarity preserving eigenvalues. Suitable shifts and deflation lead to Schur form. Complex Schur form is triangular; real Schur form can have 2×2 blocks for conjugate pairs, whose eigenvalues cannot be read individually from the diagonal. Unshifted QR need not triangularize every matrix; plane rotation is a counterexample.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Power iteration and QR eigenvalue iteration (continued)

Shifted QR:

$$A_k - \mu_k I = Q_k R_k, \qquad A_{k+1} = R_k Q_k + \mu_k I$$

can accelerate convergence. Implementations first reduce to Hessenberg form, or tridiagonal form for symmetric matrices, then use implicit QR. Hessenberg structure reduces each QR step from $O(n^3)$ to $O(n^2)$; symmetric tridiagonal iteration is cheaper. Large sparse problems often use Lanczos for Hermitian matrices or Arnoldi generally, solving a small projected eigenproblem. For a few extreme eigenvalues this can cost less than a full factorization.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Match the method to the matrix

Structure, conditioning, and the requested output guide algorithm choice.

- general dense $Ax = b$:pivoted LU;
- symmetric positive definite:Cholesky;
- overdetermined least squares:Householder QR;
- Large sparse SPD systems can use preconditioned CG.
- Large nonsymmetric sparse systems can use GMRES or related Krylov methods.
- Rectangular low-rank analysis uses SVD.
- dense nonsymmetric eigenproblem:Hessenberg QR iteration;
- Symmetric eigenproblems use tridiagonal reduction and a suitable structured eigensolver.
- singular values:bidiagonal SVD algorithms.

No algorithm is best for every problem. Hardware and blocking change constants, while mathematical structure determines which methods apply.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Vector and matrix norms

Common vector norms are

$$\|x\|_1 = \sum_i |x_i|, \qquad \|x\|_2 = \left(\sum_i |x_i|^2 \right)^{1/2}, \qquad \|x\|_\infty = \max_i |x_i|.$$

Induced matrix norm:

$$\|A\| = \max_{x \neq 0} \frac{\|Ax\|}{\|x\|}.$$

For the induced Euclidean norm,

$$\|A\|_2 = \sigma_{\max}(A).$$

Other induced norms satisfy

$$\|A\|_1 = \max_j \sum_i |a_{ij}|, \qquad \|A\|_\infty = \max_i \sum_j |a_{ij}|.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Vector and matrix norms (continued)

Frobenius norm:

$$\|A\|_F = \left(\sum_{ij} |a_{ij}|^2 \right)^{1/2}.$$

The Frobenius norm treats all entries as one long vector. It is generally not an induced operator norm.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Condition numbers

For invertible A and a consistent induced norm, define

$$\kappa(A) = \|A\|\|A^{-1}\|.$$

Right-hand side perturbation:

$$\frac{\|\Delta x\|}{\|x\|} \leq \kappa(A)\frac{\|\Delta b\|}{\|b\|}.$$

A first-order matrix-perturbation estimate is

$$\frac{\|\Delta x\|}{\|x\|} \lesssim \kappa(A)\frac{\|\Delta A\|}{\|A\|}.$$

In the Euclidean norm,

$$\kappa_2(A) = \frac{\sigma_{\max}}{\sigma_{\min}}.$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Condition numbers (continued)

For nonzero b, the right-hand-side bound follows from $\Delta x = A^{-1}\Delta b$ and $\|b\| \le \|A\|\|x\|$. For matrix-only perturbations, set $\epsilon_A = \|\Delta A\|/\|A\|$ and assume $\kappa(A)\epsilon_A < 1$. The perturbed equation gives $(I + A^{-1}\Delta A)\Delta x = -A^{-1}\Delta A x$. The Neumann-series inverse bound yields $\|\Delta x\|/\|x\| \le \kappa(A)\epsilon_A/[1 - \kappa(A)\epsilon_A]$. A large condition number describes problem sensitivity rather than an implementation defect. Scaling changes the perturbation metric being measured.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Conditioning of inversion and an exact perturbation bound

Let A be invertible and $B = A + E$, using a compatible induced norm. If $\nu = \|A^{-1}E\| < 1$, the Neumann series inverts $I + A^{-1}E$. Thus B is invertible and

$$B^{-1} = (I + A^{-1}E)^{-1}A^{-1}, \qquad B^{-1} - A^{-1} = -B^{-1}EA^{-1}.$$

Using $\|B^{-1}\| \le \|A^{-1}\|/(1-\nu)$, let $\varepsilon = \|E\|/\|A\|$. If $\kappa(A)\varepsilon < 1$,

$$\frac{\|B^{-1} - A^{-1}\|}{\|A^{-1}\|} \le \frac{\kappa(A)\varepsilon}{1 - \kappa(A)\varepsilon}.$$

The derivative of inversion is $D\,\mathrm{inv}_A[E] = -A^{-1}EA^{-1}$, not entrywise reciprocation. In the 2-norm, its relative condition number equals $\kappa_2(A)$. To attain the bound, write $A = U\Sigma V^T$ and choose $E = -\delta u_n v_n^T$, $0 < \delta < \sigma_n$. Only the smallest singular direction changes, from σ_n to $\sigma_n - \delta$, giving

$$\frac{\|(A+E)^{-1} - A^{-1}\|_2}{\|A^{-1}\|_2} = \frac{\delta}{\sigma_n - \delta} = \frac{\kappa_2(A)(\delta/\sigma_1)}{1 - \kappa_2(A)(\delta/\sigma_1)}.$$

Equality holds in the bound, and $\delta \uparrow \sigma_n$ approaches singularity. Sensitivity of a fixed-right-hand-side solution, sensitivity of the entire inverse, and backward stability of an algorithm are separate claims.

Residual and forward error

Let x be the exact solution and $\widehat{x}$ the computed one.

$$e = x - \widehat{x}, \qquad r = b - A\widehat{x} = Ae.$$

Therefore

$$e = A^{-1}r, \qquad \|e\| \le \|A^{-1}\|\|r\|.$$

A small absolute residual guarantees a small absolute error only when $\|A^{-1}\|$ is controlled.

For scalar $A = 10^{-12}$, exact $x = 1$, and computed $\widehat{x} = 1 + \delta$,

$$r = -10^{-12}\delta$$

can be tiny even when the relative solution error $|\delta|$ is substantial.

Normalized backward error:

$$\eta = \frac{\|r\|}{\|A\|\|\widehat{x}\| + \|b\|}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Residual and forward error (continued)

A small η certifies small normwise backward error for this result. One such result alone does not establish backward stability of an algorithm on all inputs. The denominator uses the computed vector rather than the unknown exact solution. If it is zero, then $b = 0$ and the residual is zero; define $\eta = 0$. Forward error still depends on conditioning.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Spectral sensitivity

If

$$A = X\Lambda X^{-1},$$

eigenvalue perturbations can be amplified by $\kappa(X)$. A normal matrix has a unitary eigenbasis, avoiding that particular basis-conditioning factor.

Nonnormal matrices can have sensitive eigenvectors and large transients even with closely grouped eigenvalues.

Singular values satisfy the absolute perturbation bound

$$|\sigma_i(A + E) - \sigma_i(A)| \leq \|E\|_2.$$

Individual singular vectors can still be sensitive near repeated values. Numerical rank therefore requires a tolerance tied to the application.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Numerical rank and truncated pseudoinverses

If the singular values have a separation

$$\sigma_1 \geq \cdots \geq \sigma_r \gg \sigma_{r+1} \geq \cdots ,$$

a common relative test retains

$$\sigma_i > \tau \sigma_1$$

as numerically nonzero. Choose τ with machine precision, dimension, and noise in mind.

Truncated pseudoinverse:

$$A_\tau^+ = V_r \operatorname{diag}(\sigma_1^{-1}, \ldots, \sigma_r^{-1}) U_r^T .$$

This gives the minimum-norm least-squares solution of the truncated model $A_\tau x = b$, and satisfies the equation exactly only if $b \in C(A_\tau)$. For $A_\tau = \operatorname{diag}(1, 0)$, $b = (0, 1)^T$, the solution is zero and the residual is b. It is not an exact inverse of the original matrix. Report the threshold, retained rank, residual, and error metric.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Stationary splittings

Choose

$$A = S - T,$$

with invertible S that is inexpensive to solve. Then

$$Sx = Tx + b.$$

Iteration:

$$Sx_{k+1} = Tx_k + b.$$

Set

$$G = S^{-1}T, \qquad c = S^{-1}b,$$

to obtain

$$x_{k+1} = Gx_k + c.$$

Subtracting the exact fixed point gives

$$x - x_{k+1} = G^{k+1}(x - x_0).$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Stationary splittings (continued)

For a unique exact solution, convergence from every initial guess is equivalent to

$$\rho(G) < 1.$$

If every initial error decays, eigenvector errors force each eigenvalue modulus below 1. Conversely, powers of a Jordan block contain polynomial factors multiplying λ^k, which tend to zero when $|\lambda| < 1$; a zero-eigenvalue block is nilpotent. Thus $G^k \to 0$. Entrywise comparisons of the splitting do not replace this spectral test.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Jacobi, Gauss-Seidel, and SOR

Use the convention $A = D + L + U$, with strictly lower and upper parts taken directly from A, and assume nonzero diagonal entries. Jacobi uses

$$S = D, \qquad T = -(L + U),$$

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right).$$

Every update uses the old iterate, allowing parallel computation.

Gauss-Seidel:

$$S = D + L, \qquad T = -U,$$

Gauss-Seidel immediately uses newly computed lower-index components.

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)} \right).$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Jacobi, Gauss-Seidel, and SOR (continued)

Strict row diagonal dominance gives $\|G_J\|_\infty < 1$ and also guarantees Gauss-Seidel convergence. SPD guarantees Gauss-Seidel convergence, but not Jacobi convergence. A 3×3 matrix with diagonal 1 and off-diagonal entries 0.9 is SPD, yet its Jacobi matrix has eigenvalue -1.8 and diverges for some initial errors.

SOR relaxes each component using the already relaxed components of the current sweep.

$$x_i^{(k+1)} = (1-\omega)x_i^{(k)} + \frac{\omega}{a_{ii}}\left(b_i - \sum_{j<i} a_{ij}x_j^{(k+1)} - \sum_{j>i} a_{ij}x_j^{(k)}\right).$$

For SPD matrices, $0 < \omega < 2$ ensures convergence, and $\omega = 1$ gives Gauss-Seidel. Averaging a completed unrelaxed GS vector with the old vector gives a different iteration. A coordinate relaxation changes energy by $-\omega(2-\omega)r_i^2/(2a_{ii})$. Each full sweep strictly reduces error energy unless the residual is zero. On the compact unit error-energy sphere, the continuous sweep map has a contraction factor strictly below 1, proving convergence to the unique minimizer.

Convergence and failure examples

For

$$A = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix},$$

the Jacobi matrix is

$$G_J = -D^{-1}(L+U) = \begin{bmatrix} 0 & -1/4 \\ -1/3 & 0 \end{bmatrix}.$$

Since

$$G_J^2 = \begin{bmatrix} 1/12 & 0 \\ 0 & 1/12 \end{bmatrix},$$

so

$$\rho(G_J) = 1/\sqrt{12} < 1,$$

so Jacobi converges.

A matrix with zero diagonal,

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Convergence and failure examples (continued)

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

cannot use Jacobi directly because of division by zero. Reordering variables can sometimes repair the diagonal, but a nonzero diagonal alone does not imply convergence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Conjugate gradients for real SPD systems

For real symmetric positive-definite A, define the A-inner product

$$\langle u, v \rangle_A = u^T A v.$$

and quadratic energy

$$\phi(x) = \frac{1}{2} x^T A x - b^T x.$$

whose unique minimum solves $Ax = b$. CG chooses A-orthogonal search directions in Krylov spaces. In exact arithmetic, it terminates in at most n steps.

Specify an initial vector x_0, defaulting to zero, nonnegative $\mathrm{atol}, \mathrm{rtol}$, and an iteration limit. Set $\tau = \max(\mathrm{atol}, \mathrm{rtol}\|b\|_2)$, and compute $r_0 = b - Ax_0$. If $\|r_0\|_2 \le \tau$, return x_0 immediately without computing any step length. Otherwise set $p_0 = r_0$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

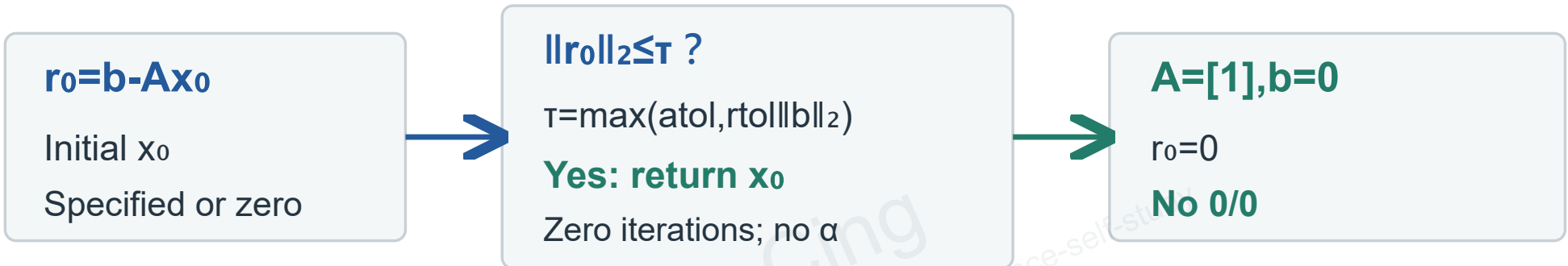
Conjugate gradients for real SPD systems (continued)

At iteration k, compute $q_k = Ap_k$, $\alpha_k = (r_k^T r_k)/(p_k^T q_k)$, then $x_{k+1} = x_k + \alpha_k p_k$, $r_{k+1} = r_k - \alpha_k q_k$. If the new residual passes tolerance, recompute $b - Ax_{k+1}$ before returning. Otherwise compute $\beta_k = (r_{k+1}^T r_{k+1})/(r_k^T r_k)$, $p_{k+1} = r_{k+1} + \beta_k p_k$. In exact arithmetic, every nonzero direction has $p_k^T A p_k > 0$. A nonpositive or nonfinite denominator in floating point signals numerical failure. If the recursive residual passes but the true residual fails, replace it with the true residual and restart with $p = r$; do not reuse a nearly zero old residual. At the iteration limit, report nonconvergence if the true residual still fails.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Conjugate gradients for real SPD systems (continued)

CG initialization: compute r_0 before iterating



Only if tolerance fails, set $p_0=r_0$ and begin iteration.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Conjugate gradients for real SPD systems (continued)

CG iteration: denominator, updates, and true residual



True pass: converged; failed at limit: iteration_limit.

Recursive pass, true fail: r ← b−Ax; p ← r (restart).

Otherwise compute β, update p, and repeat from q=Ap.

For $A=[1], b=[0], x_0=[0]$, the initial residual is zero, so return after zero iterations; a premature step calculation gives $0/0$. For $A=\begin{bmatrix}4 & 1\\ 1 & 3\end{bmatrix}$, $b=(1,2)^T$, $x_0=0$, start with $r_0=p_0=(1,2)^T$, $Ap_0=(6,7)^T$. Then $\alpha_0=5/20=1/4$, $x_1=(1/4,1/2)^T$, $r_1=(-1/2,1/4)^T$. Next $\beta_0=1/16$, $p_1=(-7/16,3/8)^T$, $\alpha_1=4/11$, and $x_2=(1/11,7/11)^T$, which satisfies $Ax_2=b$.

Conjugate gradients for real SPD systems (continued)

Exact line searches and this β_k make residuals mutually orthogonal and directions mutually A-conjugate. Nonzero conjugate directions are independent, so at most n occur. Floating point can lose orthogonality, so finite termination does not replace residual checks. Each iteration uses one matrix-vector product and a fixed number of inner products, costing $O(\mathrm{nnz}(A) + n)$ with a fixed number of length-n work vectors. No inverse is formed.

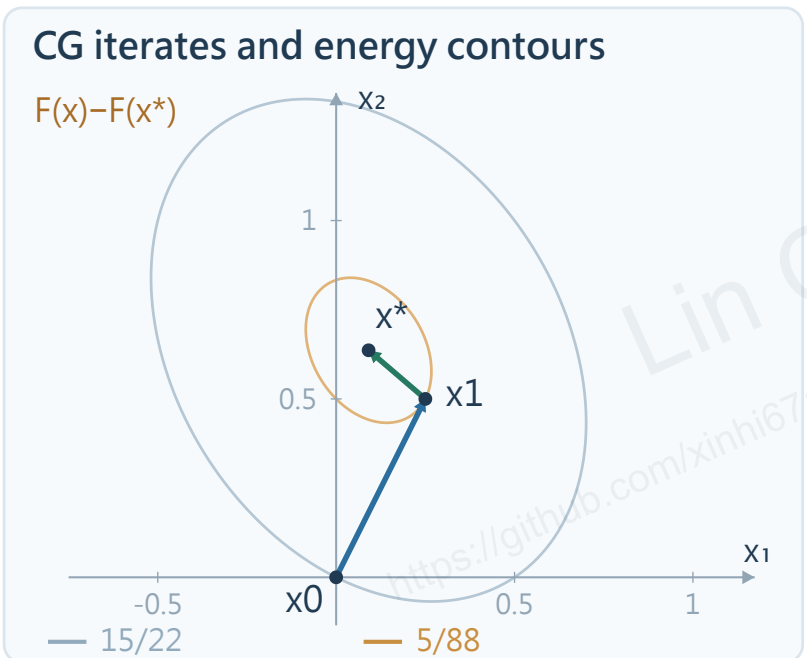
In the energy norm, the error satisfies

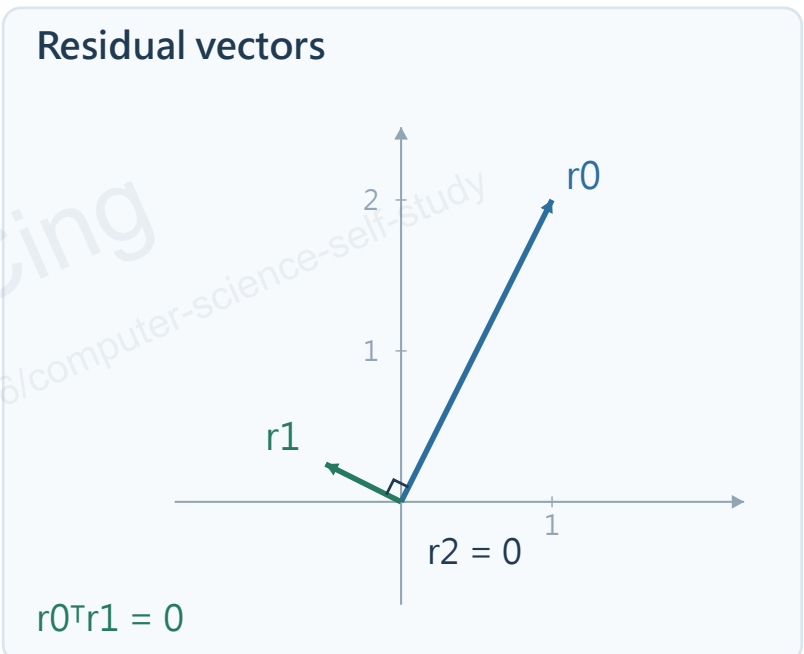
$$\|x - x_k\|_A \leq 2 \left(\frac{\sqrt{\kappa_2(A)} - 1}{\sqrt{\kappa_2(A)} + 1} \right)^k \|x - x_0\|_A.$$

Ill conditioning can slow convergence, while clustered eigenvalues often give faster convergence than this condition-number bound predicts. Standard CG requires an SPD operator and an SPD preconditioner used in a symmetry-preserving formulation.

Two geometries in the same CG example

Keep $A = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix}$, b=(1,2)^T and x_0=0. The left panel plots parameter iterates; the right plots residuals $r_k = b - Ax_k$.





Each panel has equal horizontal and vertical scales.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A-conjugacy is not Euclidean orthogonality

For $F(x) = \frac{1}{2}x^T Ax - b^T x$, the exact solution is $x_* = (1/11, 7/11)^T$.

k	x_k	r_k	$F(x_k)-F(x^*)$
0	(0,0)	(1,2)	15/22
1	(1/4,1/2)	(−1/2,1/4)	5/88
2	(1/11,7/11)	(0,0)	0

The directions are $p_0=(1,2)^T$ and $p_1=(-7/16,3/8)^T$, giving

$$p_0^T A p_1 = 0, \qquad p_0^T p_1 = 5/16 \neq 0.$$

By contrast, $r_0^T r_1 = 0$. The right angle between residuals does not imply a right angle between search segments. CG minimizes the energy error $F(x_k) - F(x_*) = \frac{1}{2}\|x_k - x_*\|_A^2$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

The residual two-norm need not decrease at every step

Take A=diag(1,100), b=(1,1/10)ᵀ and $x_0=0$. The first step gives

$$\alpha_0 = \frac{101/100}{2} = \frac{101}{200}, \quad x_1 = (101/200, 101/2000)^T, \quad r_1 = (99/200, -99/20)^T.$$

The exact solution is $x_* = (1, 1/1000)^T$.

k	$\| r_k \|_2^2$	$F(x_k)-F(x^*)$
0	101/100 = 1.01	10001/20000 = 0.50005
1	989901/40000 = 24.747525	9801/40000 = 0.245025

Energy error falls while the residual two-norm rises. This does not contradict exact-arithmetic CG theory. A true-residual stopping rule remains useful without stepwise monotonicity.

CG correctness through Krylov spaces and energy minimization

Work in exact arithmetic. Let x_* be the solution, and define $\mathcal{K}_0 = \{0\}$ and

$$\mathcal{K}_k = \operatorname{span}\{r_0, Ar_0, \ldots, A^{k-1}r_0\}.$$

At iteration k, maintain that $p_0, \ldots, p_{k-1}$ span $\mathcal{K}_k$ and are mutually A-conjugate, $x_k \in x_0 + \mathcal{K}_k$, and $r_k = b - Ax_k \perp \mathcal{K}_k$. Then x_k uniquely minimizes energy in this affine space, because for $z \in \mathcal{K}_k$,

$$\phi(x_k + z) - \phi(x_k) = -z^T r_k + \tfrac{1}{2} z^T A z = \tfrac{1}{2} z^T A z,$$

which is strictly positive for nonzero z. The invariant holds at $k = 0$. Also $r_k = r_0 - A(x_k - x_0) \in \mathcal{K}_{k+1}$. A nonzero residual is orthogonal to $\mathcal{K}_k$ and adds an independent direction.

Set $p_0 = r_0$. For $k \geq 1$, let $p_k = r_k + \beta_{k-1} p_{k-1}$. For $i \leq k - 2$, $Ap_i \in \mathcal{K}_{i+2} \subseteq \mathcal{K}_k$, so $p_i^T A r_k = (Ap_i)^T r_k = 0$. Existing conjugacy leaves only the preceding direction to cancel.

$$\beta_{k-1} = -\frac{p_{k-1}^T A r_k}{p_{k-1}^T A p_{k-1}}.$$

Since $r_k = r_{k-1} - \alpha_{k-1} A p_{k-1}$, with $r_{k-1} \in \mathcal{K}_k$,

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CG correctness through Krylov spaces and energy minimization (continued)

$$\|r_k\|^2 = -\alpha_{k-1} r_k^T A p_{k-1}.$$

substituting $\alpha_{k-1} = \|r_{k-1}\|^2 / (p_{k-1}^T A p_{k-1})$ gives $\beta_{k-1} = \|r_k\|^2 / \|r_{k-1}\|^2$. Thus the short recurrence enforces conjugacy to every earlier direction. The new direction differs from the nonzero residual only by an old direction, so it extends the span exactly to $\mathcal{K}_{k+1}$.

Line minimization gives derivative $-p_k^T r_k + \alpha p_k^T A p_k$. Since $p_k^T r_k = \|r_k\|^2$, this yields the stated α_k. After updating, $p_k^T r_{k+1} = 0$. For $i < k$,

$$p_i^T r_{k+1} = p_i^T r_k - \alpha_k p_i^T A p_k = 0.$$

so $r_{k+1} \perp \mathcal{K}_{k+1}$, preserving the invariant. Since $r_i \in \mathcal{K}_{i+1}$, residuals are mutually orthogonal. If $\sum c_i p_i = 0$, multiply by $p_j^T A$ to get $c_j p_j^T A p_j = 0$. Nonzero conjugate directions are independent, so there are at most n. If not terminated earlier, the residual at step n is orthogonal to $\mathbb{R}^n$ and must vanish.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Deriving the CG condition-number bound

Let $e_k = x_* - x_k$. Energy minimization makes CG minimize $\|e_k\|_A$ in $x_0 + \mathcal{K}_k$. Since $r_0 = Ae_0$, every available error is $p(A)e_0$ with $\deg p \le k, p(0) = 1$. Expanding in eigen-coordinates of $A = Q\Lambda Q^T$ gives

$$\|e_k\|_A \le \min_{\deg p \le k,\, p(0)=1} \max_i |p(\lambda_i)| \, \|e_0\|_A.$$

Let $a = \lambda_{\min} > 0, b = \lambda_{\max} > a$. Chebyshev polynomials satisfy $T_k(\cos\theta) = \cos(k\theta)$, hence $|T_k| \le 1$ on $[-1, 1]$. Choose

$$p(t) = \frac{T_k((b + a - 2t)/(b - a))}{T_k((b + a)/(b - a))}.$$

so $p(0) = 1$ and $[a, b]$ maps into $[-1, 1]$ in the numerator. Cosine addition gives $T_{k+1}(z) = 2zT_k(z) - T_{k-1}(z)$, with $T_0 = 1, T_1 = z$. This verifies $T_k(z) = (\rho^k + \rho^{-k})/2$ for $z > 1$, where $\rho = z + \sqrt{z^2 - 1}$. With $\kappa = b/a$, substituting $z = (b + a)/(b - a)$ gives $\rho = (\sqrt{\kappa} + 1)/(\sqrt{\kappa} - 1)$, so

$$\|e_k\|_A \le 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k \|e_0\|_A.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Deriving the CG condition-number bound (continued)

If $a = b$, then $A = aI$; a single step of length $1/a$ solves any nonzero residual. Treat this case separately rather than dividing by zero. Eigenvalue clusters permit better polynomials, so the interval bound need not be tight. The proof uses exact orthogonality; floating-point acceptance still requires the true residual.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CG for complex Hermitian positive-definite systems

This extension uses the complex inner product. The real SPD formulas with $r^T r$ must not be applied unchanged to complex matrices. If

$$A^H = A, \qquad z^H A z > 0 \quad (z \neq 0),$$

then A is Hermitian positive definite. With inner product $u^H v$, replace the real formulas by

$$r_k = b - A x_k, \qquad \alpha_k = \frac{r_k^H r_k}{p_k^H A p_k},$$

$$x_{k+1} = x_k + \alpha_k p_k, \qquad r_{k+1} = r_k - \alpha_k A p_k,$$

$$\beta_k = \frac{r_{k+1}^H r_{k+1}}{r_k^H r_k}, \qquad p_{k+1} = r_{k+1} + \beta_k p_k.$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

CG for complex Hermitian positive-definite systems (continued)

Hermitian symmetry makes $p_k^H A p_k$ real, and positive definiteness makes numerator and denominator positive for nonzero residual and direction. Use $\|r_k\|_2 = \sqrt{r_k^H r_k}$ and check r_0 first. Return x_0 immediately if it is zero. A preconditioner M must also be Hermitian positive definite and enter through a symmetry-preserving change of variables. The real proof then carries over with conjugate inner products and real positive energies and step lengths. Checking $A^T = A$ is insufficient; complex symmetric [i] has no positive real energy.

Real SPD matrices are the special case where conjugation leaves entries unchanged.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

GMRES for nonsymmetric systems

First take $A \in \mathbb{R}^{n \times n}$; complex Arnoldi uses conjugate inner products. The following Arnoldi and least-squares construction provides the algorithmic details. GMRES works in the Krylov space

$$\mathcal{K}_k(A, r_0) = \operatorname{span}\{r_0, Ar_0, \ldots, A^{k-1} r_0\}$$

and minimizes the residual.

$$\min_{x \in x_0 + \mathcal{K}_k} \|b - Ax\|_2.$$

Arnoldi starts with $v_1 = r_0 / \|r_0\|$. For $w = Av_j$, subtract $h_{ij} v_i$ successively, with $h_{ij} = v_i^T w$, and normalize using $h_{j+1,j} = \|w\|$. This yields $AV_k = V_{k+1} \overline{H}_k$. For $x = x_0 + V_k y$, the residual norm equals $\|\|r_0\| e_1 - \overline{H}_k y\|_2$; solve this small least-squares problem for the iterate. Zero initial residual finishes immediately. A zero Arnoldi remainder requires distinguishing convergence from breakdown. Restarting limits storage but can cause stagnation by discarding old directions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

GMRES for nonsymmetric systems (continued)

For nonnormal matrices, eigenvalues alone do not predict GMRES transients. The field of values, pseudospectra, and actual residual history add information. Monitor the true residual. Each unrestarted Arnoldi step uses a matrix-vector product and orthogonalization against all old directions. After $k \leq n$ steps, extra storage is $O(kn)$, and work is $O(k \operatorname{nnz}(A) + k^2 n)$. Dense matrix-vector products alone cost $O(kn^2)$. Restarted GMRES(m) limits storage to $O(mn)$, with possible stagnation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

GMRES step one: solve the reduced least-squares problem

Use the nonsymmetric invertible matrix and initial guess

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad x_0 = 0.$$

The initial residual is $r_0 = b$, with $\beta = \|r_0\|_2 = 1$, so $v_1 = e_2 = (0,1)^T$. The first Arnoldi step gives

$$w = Av_1 = (1,1)^T, \quad h_{11} = v_1^T w = 1,$$

$$w \leftarrow w - h_{11} v_1 = (1,0)^T, \quad h_{21} = 1, \quad v_2 = e_1.$$

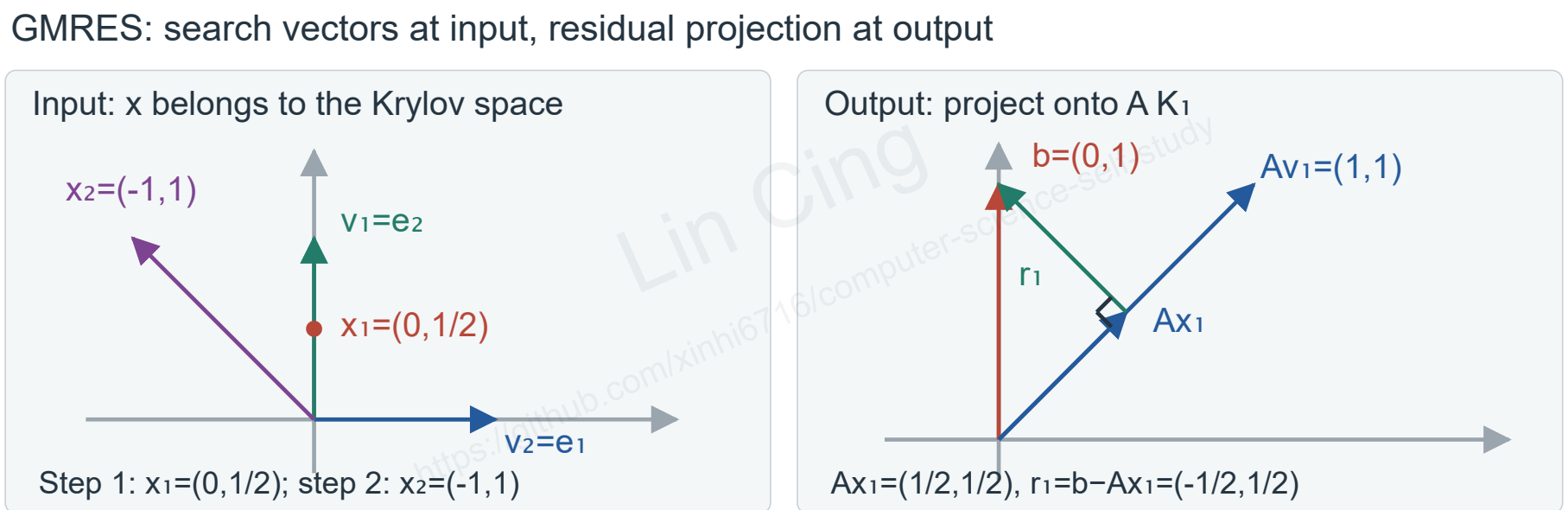
Thus $\overline{H}_1 = (1,1)^T$. Solve the one-variable least-squares problem:

$$\min_y \left\| \begin{bmatrix} 1 \\ 0 \end{bmatrix} - \begin{bmatrix} 1 \\ 1 \end{bmatrix} y \right\|_2^2 = \min_y \big((1-y)^2 + y^2 \big).$$

Its derivative is $4y - 2$, giving $y = 1/2$. Reconstruct $x_1 = V_1 y = (0, 1/2)^T$. The true residual is $r_1 = b - Ax_1 = (-1/2, 1/2)^T$, with norm $1/\sqrt{2}$. Check $(Av_1)^T r_1 = 0$: the residual is perpendicular to the **output space** $A\mathcal{K}_1$, not necessarily to the input search space $\mathcal{K}_1$.

GMRES: input search and output projection

Both spaces happen to be $\mathbb{R}^2$, but they have different roles. The left panel tracks the unknown x. The right projects b onto $A\mathcal{K}_1 = \text{span}\{(1,1)^T\}$. Each panel uses equal coordinate scales. The right-angle marker represents $(Av_1)^T r_1 = 0$.



r_1 is perpendicular to $A\,K_1$, not necessarily K_1

GMRES step two: zero Arnoldi remainder and exact termination

Now $V_2 = [v_1, v_2] = [e_2, e_1]$. Start from $w = Av_2 = e_1$:

Operation	Numerical result
$h_{12} = v_1^T w$	0; w is unchanged
$h_{22} = v_2^T w = 1$	Subtracting $h_{22}v_2$ leaves $w = (0, 0)^T$
$h_{32} = \|w\|_2$	0; do not divide by it to form v_3

The Krylov space is the whole $\mathbb{R}^2$, and $AV_2 = V_2 H_2$, where

$$H_2 = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, \qquad H_2 y = \beta e_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

The rows give $y_1 = 1$, then $y_1 + y_2 = 0$, so $y = (1, -1)^T$. Reconstruct

$$x_2 = V_2 y = e_2 - e_1 = (-1, 1)^T, \qquad Ax_2 = (0, 1)^T = b.$$

Zero true residual makes this a happy breakdown. A zero Arnoldi remainder alone does not certify convergence; verify the original residual.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Preconditioners

A useful $M \approx A$ is cheap to apply, supports parallel computation, and makes the transformed problem easier to iterate. Left preconditioning is

$$M^{-1}Ax = M^{-1}b.$$

For right preconditioning, set

$$AM^{-1}y = b, \qquad x = M^{-1}y$$

which changes variables while retaining the original residual interpretation.

Common choices include

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Preconditioners (continued)

- diagonal/Jacobi;
- block Jacobi;
- incomplete LU or incomplete Cholesky;
- algebraic multigrid;
- domain decomposition;
- physics-informed approximation.

Compare setup, iteration, and communication costs, not only iteration counts. The left-preconditioned residual is $M^{-1}(b - Ax)$, so also compute the true residual. CG requires a symmetry-preserving equivalent formulation; an arbitrary nonsymmetric $M^{-1}A$ cannot simply be treated as SPD.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stopping rules and reproducibility

Record at least the following for a solve.

1. initial residual;
2. relative residual reduction;
3. Norm and scaling conventions.
4. Maximum iterations and time limit.
5. library convergence/breakdown status;
6. final true residual;
7. Condition or forward-error estimate.
8. Deterministic settings or seed where relevant, plus a repeat check.

Evaluate backward error relative to $\|A\|\|\widehat{x}\| + \|b\|$. Rounding can make a recursively updated residual appear small while the true residual remains larger.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter review

- Pivoting affects algorithmic stability; conditioning measures sensitivity of the problem.
- Absolute residuals depend on scale; forward and backward errors answer different questions.
- A truncated pseudoinverse generally gives a least-squares solution; CG checks the initial residual before computing a step.
- Check assumptions, true residuals, and termination status for iterative solvers.

References and attribution

Lin Cing, handwritten linear algebra notes. Gilbert Strang, Introduction to Linear Algebra, 5th ed., Chapter 11.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study